



SUNWAY
UNIVERSITY

IST3134 — Big Data Analytics in the Cloud

Group Assignment Report

“Flight Delay Pattern Analysis Using PySpark on Amazon EMR”

Group Members:

Lee Zong Xun (22083844) | Bernice Wong Jian Xuan (22056998)

Group Number:

67

GitHub: <https://github.com/Ahxunnn/IST3134-Flight-Delay-Analysis>

Submission Date: 16/8/2026

Contents

1.0 Introduction to the Problem	3
2.0 Introduction to the Dataset.....	4
2.1 Dataset Source	4
2.2 Dataset Schema.....	4
2.3 Data Quality Assessment	5
3.0 Explanation of MapReduce and Spark Approach.....	6
3.1 Why We Chose Big Data Tools.....	6
3.2 Infrastructure: Amazon EMR	6
3.3 Five Scripts: Our Full Implementation	6
3.4 Key Spark Operations We Used	7
groupBy and agg	7
filter.....	7
Window Functions	7
toPandas() for Visualization	8
3.5 Physical Execution Plan.....	8
3.6 API Comparison: RDD vs DataFrame vs Spark SQL	10
3.7 Visualization Approach	12
4.0 Analysis of Output	13
4.1 Analysis 1 — Total Flights per Airline.....	13
4.2 Analysis 2 — Average Arrival Delay per Airline	14
4.3 Analysis 3 — Total Minutes Lost per Delay Cause	15
4.4 Analysis 4 — Top 10 Worst Origin Airports by Departure Delay	17
4.5 Analysis 5 — Cancellations by Reason.....	18
4.6 Analysis 6 — Monthly Average Arrival Delay Trend.....	19
4.7 Analysis 7 — Top 3 Most Delayed Routes per Airline (Window Function)	20
4.8 Year-on-Year Comparison: 2017 vs 2018	22
5.0 Individual Reflection	25
5.1 Lee Zong Xun 22083844	25
5.2 Bernice Wong Jian Xuan 22056998	25
6.0 References.....	27

1.0 Introduction to the Problem

Flight delays are one of the most persistent and frustrating challenges in the United States aviation industry. Anyone who has waited at an airport gate for hours knows the feeling — and behind those individual experiences lies a massive economic problem. According to the Bureau of Transportation Statistics (BTS), delayed and cancelled flights cost airlines, airports, and passengers billions of dollars each year through missed connections, hotel bookings, lost productivity, and wasted fuel.

What makes flight delays particularly interesting from a data perspective is how they cascade. A single aircraft running late in the morning does not just affect one flight — it ripples through every subsequent rotation that aircraft is scheduled to make throughout the day, potentially disrupting thousands of passengers across multiple routes before the day is over.

In this project, we set out to analyse domestic US flight delay and cancellation patterns across 2017 and 2018 using Apache Spark on Amazon EMR. We focused on three core research questions that we felt would produce the most actionable insights:

1. Which airlines have the highest average arrival delays, and how do their delay patterns differ?
2. Which origin airports consistently produce the worst departure delays, and what explains this?
3. What actually causes the most total delay minutes — weather, airline operations, air traffic control, or the cascading effect of late aircraft?

We chose this topic because the answers are not always what people expect. Most passengers assume weather is the dominant cause of delays, but as our analysis reveals, the reality is considerably more nuanced. At nearly 13 million flight records, this dataset is far too large for conventional single-machine tools, making it an ideal candidate for distributed Big Data processing on Amazon EMR.

2.0 Introduction to the Dataset

2.1 Dataset Source

The dataset we used is the US Airline Delay and Cancellation Data (2009–2018), published by the Bureau of Transportation Statistics and made available on Kaggle at:

<https://www.kaggle.com/datasets/yuanyuwendymu/airline-delay-and-cancellation-data-2009-2018>

We chose to work with 2017 and 2018 data only, which gave us 12,888,067 flight records — approximately 700 MB per year, totalling around 1.4 GB of raw CSV data. This two-year window is large enough to identify meaningful patterns while remaining manageable within our AWS Academy Learner Lab credit budget. To put the scale in perspective, trying to load both files into a standard pandas DataFrame on a single machine would require over 8 GB of RAM just for the data alone, before any processing begins. PySpark on EMR distributes this work across the cluster, making what would otherwise be impractical into a routine operation.

2.2 Dataset Schema

The dataset contains 27 columns organised into three logical groups. The table below summarises each column, its data type as inferred by Spark, and what it represents in practice:

Column	Data Type	Description
FL DATE	Date	Date the flight operated
OP CARRIER	String	Airline IATA code (e.g. WN = Southwest, DL = Delta)
OP CARRIER FL NUM	Integer	Flight number assigned by the airline
ORIGIN	String	Departure airport 3-letter IATA code
DEST	String	Arrival airport 3-letter IATA code
CRS DEP TIME	Integer	Planned departure time (HHMM format)
DEP TIME	Float	Actual departure time
DEP DELAY	Float	Departure delay in minutes (negative = early)
TAXI OUT	Float	Minutes from gate departure to wheels leaving ground
WHEELS OFF	Float	Time aircraft left the ground
WHEELS ON	Float	Time aircraft touched down
TAXI IN	Float	Minutes from wheels-on to gate arrival
CRS ARR TIME	Integer	Planned arrival time
ARR TIME	Float	Actual arrival time
ARR DELAY	Float	Arrival delay in minutes — primary analysis metric
CANCELLED	Float	Flight cancelled flag (1 = cancelled, 0 = operated)
CANCELLATION CODE	String	A=Carrier, B=Weather, C=NAS, D=Security
DIVERTED	Float	Aircraft landed at unscheduled airport (1 = yes)
CRS ELAPSED TIME	Float	Planned total flight duration in minutes
ACTUAL ELAPSED TIME	Float	Actual total flight duration in minutes
AIR TIME	Float	Time airborne (wheels-off to wheels-on)
DISTANCE	Float	Distance between airports in miles
CARRIER DELAY	Float	Minutes delayed due to airline operations

WEATHER_DELAY	Float	Minutes delayed due to extreme weather only
NAS_DELAY	Float	Minutes delayed due to air traffic control / NAS
SECURITY_DELAY	Float	Minutes delayed due to security incidents
LATE_AIRCRAFT_DELAY	Float	Minutes delayed due to late inbound aircraft

One column worth highlighting is `ARR_DELAY`, this is the primary metric BTS uses for official on-time performance reporting, and it became the central focus of most of our analyses. A flight is officially classified as delayed only when `ARR_DELAY` reaches 15 minutes or more; anything less is counted as on time in official statistics even if the plane arrived a few minutes late.

2.3 Data Quality Assessment

Before diving into the analysis, we ran a thorough data quality check across all 27 columns. The results were illuminating — not because the data was poor quality, but because understanding why certain columns have missing values is essential to interpreting the results correctly.

The flight status breakdown across all 12,888,067 records is as follows:

Status	Count	Percentage
Early arrivals (<code>ARR_DELAY < 0</code>)	7,944,356	61.6%
On time (<code>ARR_DELAY = 0</code>)	250,863	1.9%
Delayed (<code>ARR_DELAY > 0</code>)	4,460,597	34.6%
Null <code>ARR_DELAY</code> (cancelled/diverted)	232,251	1.8%
Total	12,888,067	100%

The first thing that surprised us here was that 61.6% of all flights actually arrived early. This happens because airlines deliberately pad their published schedules with buffer time to absorb expected delays, so technically arriving "on time" often means arriving 10 or 15 minutes ahead of the planned schedule.

Regarding null values: the five delay cause columns (`CARRIER_DELAY`, `WEATHER_DELAY`, `NAS_DELAY`, `SECURITY_DELAY`, `LATE_AIRCRAFT_DELAY`) each have around 10.5 million null values. This is not a data collection failure, it is because BTS only requires airlines to report delay reasons for flights delayed 15 minutes or more. Shorter delays are recorded in `ARR_DELAY` but carry no cause attribution. This means our delay cause analysis in Section 4.3 represents only the subset of delays meeting the 15-minute reporting threshold, which is an important point to keep in mind. Of 4,460,597 delayed flights, 2,078,414 (46.6%) fall below this threshold and have no cause breakdown recorded.

We also discovered a completely empty trailing column (Unnamed: 27) present in both CSV files, since this is completely useless and meaningless, we decide to exclude it from all analyses.

3.0 Explanation of MapReduce and Spark Approach

3.1 Why We Chose Big Data Tools

The answer is simple; the dataset simply would not fit in memory on a single machine. At 12,888,067 rows across approximately 1.4 GB of raw CSV, loading both files into pandas would require the entire dataset to reside in RAM simultaneously. On a typical laptop with 8 or 16 GB of RAM shared with the operating system and other applications, this will either result in a complete failure or cause the system to respond so slowly that it is impossible to complete any analysis within a reasonable amount of time.

Apache Spark solves this by splitting the dataset into partitions and distributing them across the nodes in our EMR cluster. Each node processes its own slice of the data independently, and results are combined through network shuffles only when necessary. For operations like groupBy aggregation, Spark performs a partial aggregation on each node first, this reducing data volume before any network transfer, which is the same two-phase approach MapReduce pioneered. The difference is that Spark keeps intermediate results in memory rather than writing to disk between stages, making it far faster for iterative analytical queries.

3.2 Infrastructure: Amazon EMR

We ran all analysis on an Amazon EMR cluster configured as follows:

Component	Specification
EMR Release	emr-7.x (latest)
Primary Node	1× m5.xlarge — 4 vCPU, 16 GB RAM — runs NameNode, ResourceManager
Core Nodes	2× m5.xlarge — 4 vCPU, 16 GB RAM each — run DataNode, NodeManager
Framework	Apache Spark 3.5.6-amzn-2, Hadoop 3.x, YARN, Python 3.11
Storage	HDFS for active processing, Amazon S3 for persistent backup
DataFrame API Duration	~3 minutes across 12,888,067 records
Spark SQL Duration	~2 minutes across 12,888,067 records
RDD API Duration	2 minutes 21 seconds across 12,888,067 records

There is one lesson we learned the hard way: when our first EMR cluster terminated unexpectedly, all HDFS data was lost along with it. This taught us to always back up outputs to S3 immediately after processing. Unlike HDFS, which lives only on the cluster nodes, S3 persists independently of any cluster, making it the appropriate destination for any results we want to keep beyond a single session.

3.3 Five Scripts: Our Full Implementation

We developed five Python scripts in total, covering the complete pipeline from raw analysis through to visual output:

Script	API / Tool	Purpose	Output
analysis_pyspark.py	DataFrame API	7 analyses — flights, delays, airports, cancellations, monthly trend, routes	Console
analysis_sql.py	Spark SQL	Same 7 analyses using SQL syntax — identical results	Console
analysis_rdd.py	RDD API	3 analyses at low level — demonstrates full Spark stack	Console
visualizations.py	DataFrame + matplotlib	7 charts generated from aggregated results	PNG files
viz_yoy.py	DataFrame + matplotlib	3 year-on-year comparison charts	PNG files

3.4 Key Spark Operations We Used

groupBy and agg

The workhorse of most of our analyses. We used this to compute aggregations per airline, per airport, per month, and per cancellation code:

```
# Filter only delayed flights, then group and aggregate
df.filter(F.col("ARR_DELAY") > 0)
  .groupBy("OP_CARRIER")
  .agg(
    F.count("*").alias("delayed_flights"),
    F.round(F.avg("ARR_DELAY"), 2).alias("avg_delay_min"),
    F.round(F.max("ARR_DELAY"), 2).alias("max_delay_min")
  )
  .orderBy(F.desc("avg_delay_min"))
```

filter

Used extensively to restrict rows before aggregation, which reduces shuffle volume and speeds up computation:

```
df.filter(F.col("ARR_DELAY") > 0)    # delayed flights only
df.filter(F.col("CANCELLED") == 1)  # cancelled flights only
df.filter(F.col("delayed_departures") > 1000) # high-volume airports only
```

Window Functions

We used window functions in Analysis 7 to rank the three most delayed routes within each airline without collapsing the data. This is the key distinction from groupBy — window functions add a computed column while preserving every original row:

```
# Partition by airline, rank routes by delay within each airline
w = Window.partitionBy("OP_CARRIER").orderBy(F.desc("avg_delay_min"))
route_delay.withColumn("rank", F.row_number().over(w))
  .filter(F.col("rank") <= 3)
```

toPandas() for Visualization

After each Spark aggregation, we converted the small result DataFrame to pandas for charting. This is safe because `groupBy` has already reduced 12.8 million rows to 10–30 summary rows before any conversion happens:

```
# Only convert AFTER aggregation — never on raw data
data = df.groupBy("OP CARRIER")
        .agg(F.count("*").alias("total_flights"))
        .limit(10)
        .toPandas()
```

3.5 Physical Execution Plan

Running `avg_delay.explain(mode="formatted")` gave us a detailed look at how Spark actually executes the average delay query under the hood. The plan reads bottom to top:

```
== Physical Plan ==
AdaptiveSparkPlan (8)
+- Sort (7)
   +- Exchange (6)
      +- HashAggregate (5)
         +- Exchange (4)
            +- HashAggregate (3)
               +- Filter (2)
                  +- Scan csv (1)
```

Figure 1: Physical plan tree structure for the average arrival delay query (read bottom to top).

```

(1) Scan csv
Output [2]: [OP_CARRIER#19, ARR_DELAY#32]
Batched: false
Location: InMemoryFileIndex [hdfs://ip-172-31-66-219.ec2.internal:8020/user/hadoop/flights]
PushedFilters: [IsNotNull(ARR_DELAY), GreaterThan(ARR_DELAY,0.0)]
ReadSchema: struct<OP_CARRIER:string,ARR_DELAY:double>

(2) Filter
Input [2]: [OP_CARRIER#19, ARR_DELAY#32]
Condition : (isNotNull(ARR_DELAY#32) AND (ARR_DELAY#32 > 0.0))

(3) HashAggregate
Input [2]: [OP_CARRIER#19, ARR_DELAY#32]
Keys [1]: [OP_CARRIER#19]
Functions [2]: [partial_count(1), partial_avg(ARR_DELAY#32)]
Aggregate Attributes [3]: [count#110L, sum#112, count#113L]
Results [4]: [OP_CARRIER#19, count#111L, sum#114, count#115L]

(4) Exchange
Input [4]: [OP_CARRIER#19, count#111L, sum#114, count#115L]
Arguments: hashpartitioning(OP_CARRIER#19, 1000), ENSURE_REQUIREMENTS, [plan_id=43]

(5) HashAggregate
Input [4]: [OP_CARRIER#19, count#111L, sum#114, count#115L]
Keys [1]: [OP_CARRIER#19]
Functions [2]: [count(1), avg(ARR_DELAY#32)]
Aggregate Attributes [2]: [count(1)#103L, avg(ARR_DELAY#32)#105]
Results [3]: [OP_CARRIER#19, count(1)#103L AS delayed_flights#104L, round(avg(ARR_DELAY#32)#105, 2) AS avg_delay_min#106]

(6) Exchange
Input [3]: [OP_CARRIER#19, delayed_flights#104L, avg_delay_min#106]
Arguments: rangepartitioning(avg_delay_min#106 DESC NULLS LAST, 1000), ENSURE_REQUIREMENTS, [plan_id=46]

(7) Sort
Input [3]: [OP_CARRIER#19, delayed_flights#104L, avg_delay_min#106]
Arguments: [avg_delay_min#106 DESC NULLS LAST], true, 0

(8) AdaptiveSparkPlan
Output [3]: [OP_CARRIER#19, delayed_flights#104L, avg_delay_min#106]
Arguments: isFinalPlan=false

```

Figure2: Detailed physical plan showing column pruning, predicate pushdown, and shuffle operations at each stage.

Looking at the plan more closely, a few things stand out. The Scan csv stage at the bottom shows that Spark was smart enough to read only 2 columns out of 27 — OP_CARRIER and ARR_DELAY — and completely skipped the remaining 25. This is called column pruning, and it means Spark is not wasting time reading data it does not need across 12.8 million rows. Right above that, the Filter stage runs immediately after the scan, so any flight that was not delayed gets thrown out before any data moves across the network. This keeps the shuffle as small as possible.

The two Exchange nodes are where data physically moves between machines — these are the most expensive steps in the entire plan. The first Exchange groups all records by airline code so that every flight belonging to the same airline ends up on the same machine for the final count and average calculation. The second Exchange then rearranges the results globally so they come out in the correct sorted order. Finally, the AdaptiveSparkPlan wrapper at the top with isFinalPlan=false means Spark is not locked into this plan — it monitors what is actually happening at runtime and can adjust. In our case, it detected that the grouped result was tiny and automatically reduced the number of output partitions from the default 1,000 down to just a handful, avoiding unnecessary overhead.

3.6 API Comparison: RDD vs DataFrame vs Spark SQL

One of the most valuable exercises in this project was implementing the same analyses across all three levels of the Spark API stack. Doing this gave us a concrete understanding of what each layer actually does and what the Catalyst optimizer provides. The same total flights query expressed across all three APIs makes the differences clear:

Layer 1 — RDD API (lowest level, no optimizer):

```
# Must manually parse CSV, strip whitespace, handle all types
# No column pruning — reads entire CSV line by line
parsed.map(lambda f: (f[carrier_idx].strip(), 1))
      .reduceByKey(lambda a, b: a + b)
      .sortBy(lambda x: -x[1])
```

Layer 2 — DataFrame API (structured, Catalyst optimized):

```
# Schema-aware — column pruning and predicate pushdown automatic
# Catalyst optimizer generates same physical plan as Spark SQL
df.groupBy("OP_CARRIER")
  .agg(F.count("*").alias("total_flights"))
  .orderBy(F.desc("total_flights"))
```

Layer 3 — Spark SQL (declarative, same plan as DataFrame):

```
-- Register DataFrame as temp view first
-- df.createOrReplaceTempView("flights")
SELECT OP_CARRIER, COUNT(*) AS total_flights
FROM flights
GROUP BY OP_CARRIER
ORDER BY total_flights DESC
```

The results table from all three implementations confirms they produce identical output:

```
=====
>>> print("IMPLEMENTATION 1: DataFrame API")
IMPLEMENTATION 1: DataFrame API
>>> print("=" * 55)
=====
>>> df.groupBy("OP_CARRIER") \
...   .agg(F.count("*").alias("total_flights")) \
...   .orderBy(F.desc("total_flights")) \
...   .show(5)
26/08/15 07:13:58 WARN SparkStringUtils: Truncated the string representation of a long (16 digits)
+-----+-----+
|OP_CARRIER|total_flights|
+-----+-----+
|WN|2681996|
|DL|1872843|
|AA|1813166|
|OO|1480664|
|UA|1206046|
+-----+-----+
only showing top 5 rows

=====
>>> print("IMPLEMENTATION 2: Spark SQL")
IMPLEMENTATION 2: Spark SQL
>>> print("=" * 55)
=====
>>> df.createOrReplaceTempView("flights")
>>> spark.sql("""
...   SELECT OP_CARRIER, COUNT(*) AS total_flights
...   FROM flights
...   GROUP BY OP_CARRIER
...   ORDER BY total_flights DESC
...   LIMIT 5
... """).show()
+-----+-----+
|OP_CARRIER|total_flights|
+-----+-----+
|WN|2681996|
|DL|1872843|
|AA|1813166|
|OO|1480664|
|UA|1206046|
+-----+-----+
```

Figure 3: DataFrame API (left) and Spark SQL (right) producing identical top 5 results, confirming both compile to the same physical plan via the Catalyst optimizer.

RDD ANALYSIS 1: Total Flights per Airline		
=====		
Carrier	Total Flights	
WN	2,681,996	
DL	1,872,843	
AA	1,813,166	
OO	1,480,664	
UA	1,206,046	
B6	603,664	
EV	542,431	
AS	430,829	
NK	332,996	
YX	316,090	
=====		

Figure 4: RDD API — Total Flights per Airline — identical to DataFrame and SQL results.

RDD ANALYSIS 2: Average Arrival Delay per Airline		
=====		
Carrier	Avg Delay (min)	
EV	51.71	
9E	49.17	
B6	48.18	
F9	46.95	
G4	44.93	
OO	44.82	
NK	43.92	
UA	42.00	
YV	41.81	
YX	38.99	
=====		

Figure 5: RDD API — Average Arrival Delay per Airline. EV (ExpressJet) worst at 51.71 min — identical to DataFrame and SQL results.

RDD ANALYSIS 3: Cancellations by Reason		
=====		
Code	Reason	Count
B	Weather	110,443
A	Carrier	48,086
C	National Air System	40,385
D	Security	363
RDD analysis complete.		

Figure 6: RDD API — Cancellations by Reason. Weather (B) leads at 110,443 — identical to DataFrame and SQL results.

Dimension	RDD API	DataFrame API	Spark SQL
Code length	~60 lines	~30 lines	~25 lines
Execution time	2 min 21 sec	~3 minutes	~2 minutes
Catalyst optimizer	None — fully manual	Full support	Full support
Column pruning	No — reads all columns	Yes — automatic	Yes — automatic
Predicate pushdown	No — filters after read	Yes — filters during read	Yes — filters during read
Best for	Custom low-level logic	Programmatic pipelines	SQL-familiar teams
Results correct?	Yes	Yes	Yes

The timing differences between the three implementations are mostly down to how YARN allocated resources between sessions rather than any real speed advantage of one API over another. At this dataset size the gap is small, but at larger scale, let's say 100 GB or 1 TB, the DataFrame API and Spark SQL would pull ahead significantly. This is because the Catalyst optimizer automatically reads only the columns it needs and filters out irrelevant rows before any data moves across the network, while the RDD API has no such optimisation and must read every column of every row from start to finish regardless of what the query actually needs.

For further similar task, we would recommend the DataFrame API or Spark SQL as the default choice. We also included the RDD implementation here specifically to show the full Spark API stack from low to high level, and to make visible exactly what the optimizer is doing behind the scenes when we are using the higher-level APIs.

3.7 Visualization Approach

To make our findings more accessible and visually compelling, we built two additional scripts that generate charts directly from PySpark aggregation results using matplotlib. The approach follows a consistent pattern across all charts: run the Spark aggregation, call `.toPandas()` on the small result, then render the chart using matplotlib's non-interactive Agg backend — required because the EMR master node has no display server attached.

We generated ten charts in total: seven covering the main analyses and three dedicated year-on-year comparison charts. All charts were saved as PNG files at 150 DPI on the EMR master node, copied to S3 using `aws s3 cp` for persistent storage, and downloaded for inclusion in this report and the GitHub repository.

4.0 Analysis of Output

All results presented in this section were produced by running `analysis_pyspark.py` and `analysis_sql.py` against the full 12,888,067-record dataset on our EMR cluster. Where both scripts were run on the same analysis, the results were identical — confirming the equivalence of the DataFrame API and Spark SQL approaches.

4.1 Analysis 1 — Total Flights per Airline

ANALYSIS 1: Total Flights per Airline

```
=====
+-----+-----+
|OP_CARRIER|total_flights|
+-----+-----+
|      WN   |    2681996   |
|      DL   |    1872843   |
|      AA   |    1813166   |
|      OO   |    1480664   |
|      UA   |    1206046   |
|      B6   |     603664   |
|      EV   |     542431   |
|      AS   |     430829   |
|      NK   |     332996   |
|      YX   |     316090   |
+-----+-----+
```

Carrier	Airline Name	Total Flights
WN	Southwest Airlines	2,681,996
DL	Delta Air Lines	1,872,843
AA	American Airlines	1,813,166
OO	SkyWest Airlines	1,480,664
UA	United Airlines	1,206,046
B6	JetBlue Airways	603,664
EV	ExpressJet Airlines	542,431
AS	Alaska Airlines	430,829
NK	Spirit Airlines	332,996
YX	Republic Airways	316,090

Southwest Airlines (WN) dominated with 2,681,996 flights — more than 800,000 ahead of second-place Delta. This is easy to understand once you look at Southwest's business model: rather than routing passengers through major hub airports the way Delta and American do, Southwest operates a point-to-point network with many short, frequent routes between secondary cities. SkyWest (OO) appearing fourth is initially surprising for a regional carrier but reflects their unique position as a contract operator, the company operates flights under the “Delta Connect,” “United Express,” and “US Airways” brands — every flight they operate is counted under their own code.

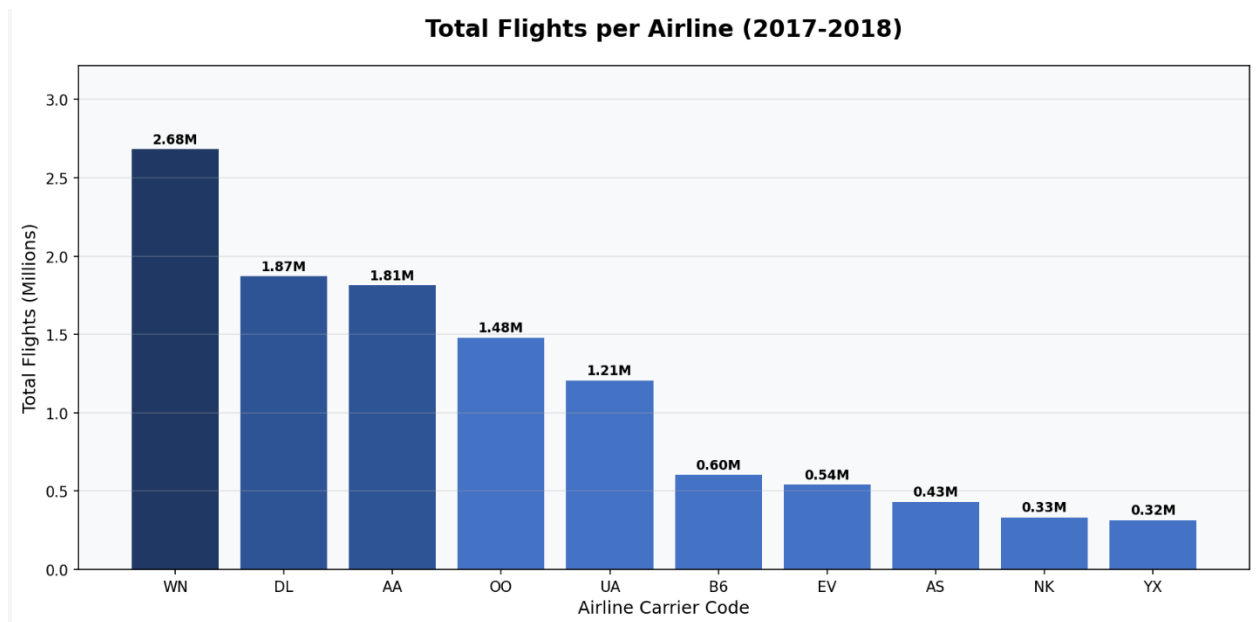


Figure 7: Total flights per airline (2017-2018). Southwest Airlines (WN) leads with 2.68 million flights, over 800,000 more than second-place Delta.

4.2 Analysis 2 — Average Arrival Delay per Airline

ANALYSIS 2: Average Arrival Delay per Airline (delayed only)

```
=====
+-----+-----+-----+-----+
|OP_CARRIER|delayed_flights|avg_delay_min|max_delay_min|
+-----+-----+-----+-----+
|EV|178190|51.71|1810.0|
|9E|73316|49.17|1685.0|
|B6|242967|48.18|1473.0|
|F9|88973|46.95|1253.0|
|G4|37177|44.93|1505.0|
|OO|504425|44.82|2692.0|
|NK|107788|43.92|1619.0|
|UA|384307|42.0|1539.0|
|YV|79640|41.81|1773.0|
|YX|100386|38.99|1346.0|
+-----+-----+-----+-----+
```

Carrier	Airline	Delayed Flights	Avg Delay (min)	Max Delay (min)
EV	ExpressJet	178,190	51.71	1,810
9E	Endeavor Air	73,316	49.17	1,685
B6	JetBlue	242,967	48.18	1,473
F9	Frontier	88,973	46.95	1,253
G4	Allegiant Air	37,177	44.93	1,505
OO	SkyWest	504,425	44.82	2,692
NK	Spirit Airlines	107,788	43.92	1,619
UA	United Airlines	384,307	42.00	1,539
YV	Mesa Airlines	79,640	41.81	1,773
YX	Republic Airways	100,386	38.99	1,346

ExpressJet (EV) came out as the worst performer, with an average delay of 51.71 minutes across 178,190 delayed flights. What we found notable here is that regional carriers dominate the worst performers — EV, 9E, OO, and YV are all regional operators. This makes structural sense because of the regional carriers typically operate with fewer spare aircraft, so when one breaks down or a crew runs into scheduling issues, there is no backup sitting at the gate. This domino effect can disrupt flight operations throughout the day. The extreme outlier is SkyWest (OO), with the longest delay of 2,692 minutes—just under 45 hours, it is the longest single delay in the entire 12.8 million record dataset. This almost certainly represents a situation where a backup aircraft had to be dispatched overnight from another city.

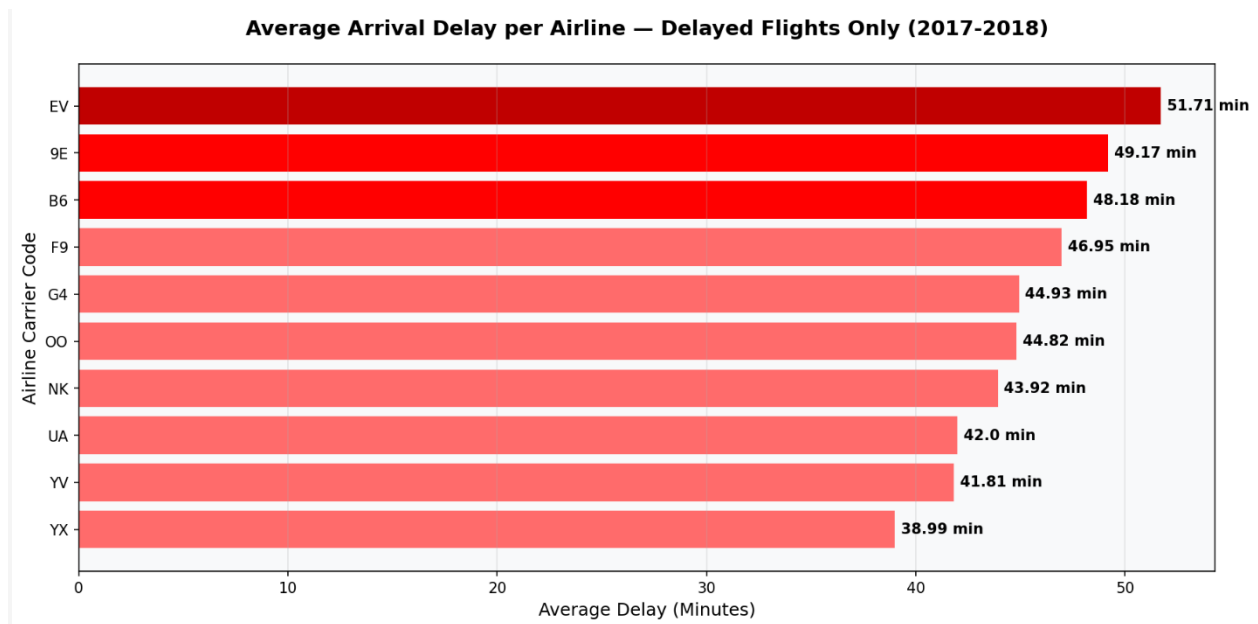


Figure 8: Average arrival delay per airline for delayed flights only. ExpressJet (EV) records the worst performance at 51.71 minutes average.

4.3 Analysis 3 — Total Minutes Lost per Delay Cause

ANALYSIS 3: Total Minutes Lost per Delay Cause

```
=====
+-----+-----+-----+-----+-----+
|carrier_delay_min|weather_delay_min|nas_delay_min|security_delay_min|late_aircraft_delay_min|
+-----+-----+-----+-----+-----+
| 4.6833683E7| 7718834.0| 3.7989222E7| 220701.0| 6.0594128E7|
+-----+-----+-----+-----+-----+
```

Delay Cause	Total Minutes Lost	% of Total
Late Aircraft (cascading)	60,594,128	39.9%
Carrier (airline operations)	46,833,683	30.8%
NAS (air traffic control)	37,989,222	25.0%
Weather (extreme only)	7,718,834	5.1%
Security	220,701	0.1%
TOTAL	153,356,568	100%

This was perhaps our most surprising finding. Late aircraft cascading delays account for 39.9% of all delay minutes — over 60 million minutes of passengers waiting across just two years. Before we ran this analysis, our expectation was that weather would be the dominant cause since it is what airlines and passengers most commonly blame. In reality, weather accounts for only 5.1% of total delay minutes. The explanation lies in how BTS categorises delays: only extreme weather events like hurricanes, blizzards, and tornadoes qualify as weather delay. Everything else like fog, light rain, moderate thunderstorms — gets classified as NAS delay instead. This is why NAS accounts for a substantial 24.8% despite weather appearing small. The practical takeaway is that if airlines want to reduce total delay minutes, addressing their own operational efficiency and the cascading late-aircraft problem would have far more impact than anything weather-related.

Total Delay Minutes by Cause (2017-2018)

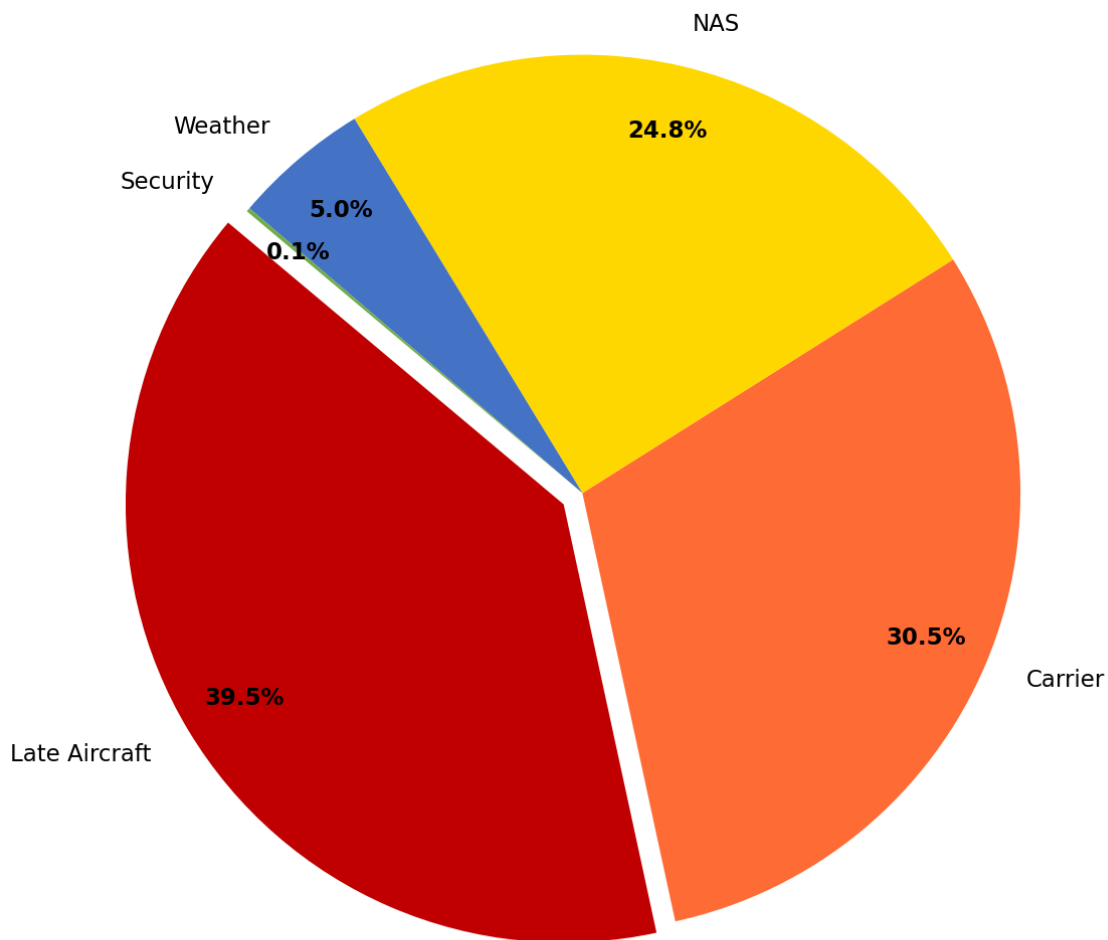


Figure 9: Total delay minutes by cause (2017-2018). Late aircraft cascading delays dominate at 39.9%, while weather accounts for only 5.1%.

4.4 Analysis 4 — Top 10 Worst Origin Airports by Departure Delay

ANALYSIS 4: Top 10 Worst Origin Airports by Avg Departure Delay

```
=====
+-----+-----+-----+
|ORIGIN|delayed_departures|avg_dep_delay_min|
+-----+-----+-----+
| EGE |      1108 |      82.95 |
| DLH |      1155 |      74.76 |
| ASE |      3610 |      72.6 |
| MLU |      1202 |      70.48 |
| AZO |      1013 |      67.2 |
| RDM |      2001 |      65.57 |
| BGR |      1347 |      65.43 |
| BTR |      3672 |      65.01 |
| AEX |      1079 |      64.75 |
| FWA |      3366 |      64.75 |
+-----+-----+-----+
```

Airport	Location	Delayed Departures	Avg Dep Delay (min)
EGE	Eagle County Regional, Colorado	1,108	82.95
DLH	Duluth International, Minnesota	1,155	74.76
ASE	Aspen/Pitkin County, Colorado	3,610	72.60
MLU	Monroe Regional, Louisiana	1,202	70.48
AZO	Kalamazoo/Battle Creek, Michigan	1,013	67.20
RDM	Roberts Field, Oregon	2,001	65.57
BGR	Bangor International, Maine	1,347	65.43
BTR	Baton Rouge, Louisiana	3,672	65.01
AEX	Alexandria International, Louisiana	1,079	64.75
FWA	Fort Wayne, Indiana	3,366	64.75

When we first saw this table, we expected to see airports like JFK, LAX, or O'Hare at the top. Instead, the worst performers are almost all small regional airports. Eagle County Regional (EGE) in Colorado tops the list at 82.95 minutes average departure delay. Based on our research, EGE serves the ski resort town of Vail, sits at high altitude in the Rocky Mountains, has a single runway, and receives infrequent service, so when one flight is delayed there, recovery is slow because there are no spare aircraft or crew stationed nearby. Aspen (ASE) ranked third for exactly the same reasons. The three Louisiana airports (MLU, BTR, AEX) reflect the Gulf Coast's vulnerability to thunderstorms and hurricane-season weather. We applied a filter requiring at least 1,000 delayed departures to ensure we were comparing airports with meaningful traffic volume rather than tiny airfields where a single weather event could affect the average.

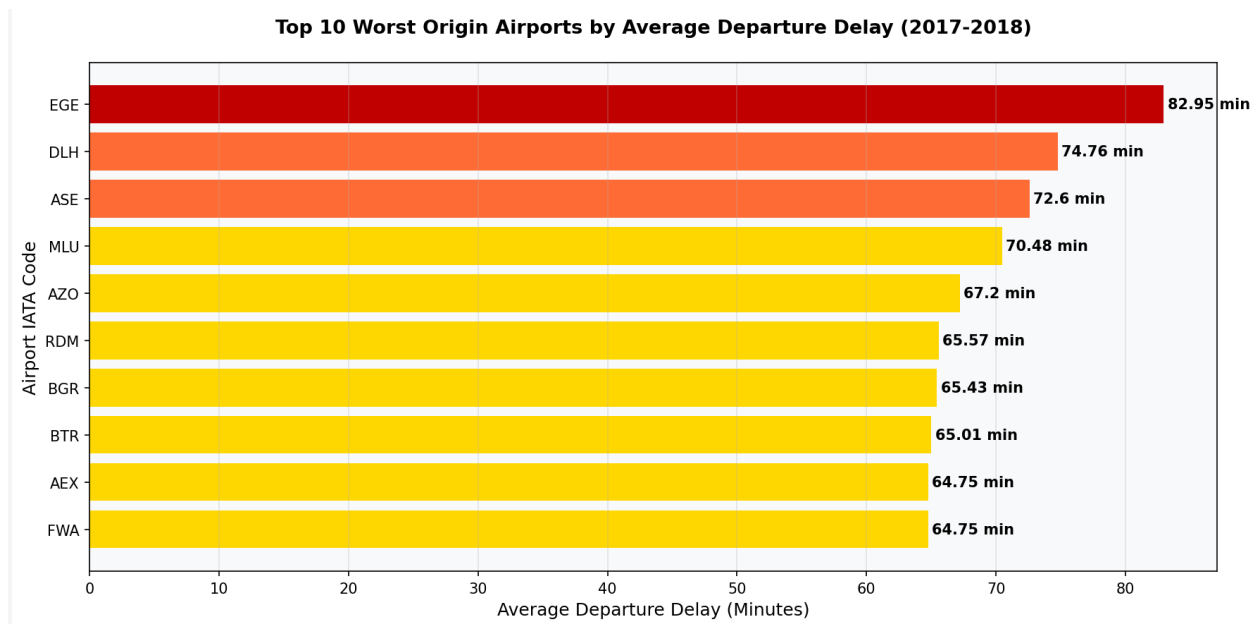


Figure 10: Top 10 worst origin airports by average departure delay. Small regional airports consistently outperform major hubs for worst delays.

4.5 Analysis 5 — Cancellations by Reason

ANALYSIS 5: Cancellations by Reason

```
=====
+-----+-----+
| CANCELLATION_CODE | cancelled_flights |
+-----+-----+
| B | 110443 |
| A | 48086 |
| C | 40385 |
| D | 363 |
+-----+-----+
```

Code	Reason	Cancelled Flights	% of Total
B	Weather	110,443	55.4%
A	Carrier / Airline	48,086	24.1%
C	National Air System	40,385	20.3%
D	Security	363	0.2%
—	TOTAL	199,277	100%

The overall cancellation rate was 1.55% — 199,277 flights out of 12,888,067. Weather (Code B) dominates cancellations at 55.4%, which creates an interesting contrast with Analysis 3 where weather contributed only 5.1% of delay minutes. But when we think carefully, this discrepancy actually makes sense, the moderate weather delays a flight by 30 or 60 minutes, while only truly severe weather, the kind that makes operating the flight genuinely unsafe, only leads to flight cancellations. Carrier cancellations (Code A, 24.1%) reflect mechanical failures and crew scheduling issues. NAS cancellations (Code C, 20.3%) happen during system-wide ground stops or airport closures coordinated by air traffic control. Security cancellations (Code D) are very rare — just 363 flights across two full years of US domestic operations.

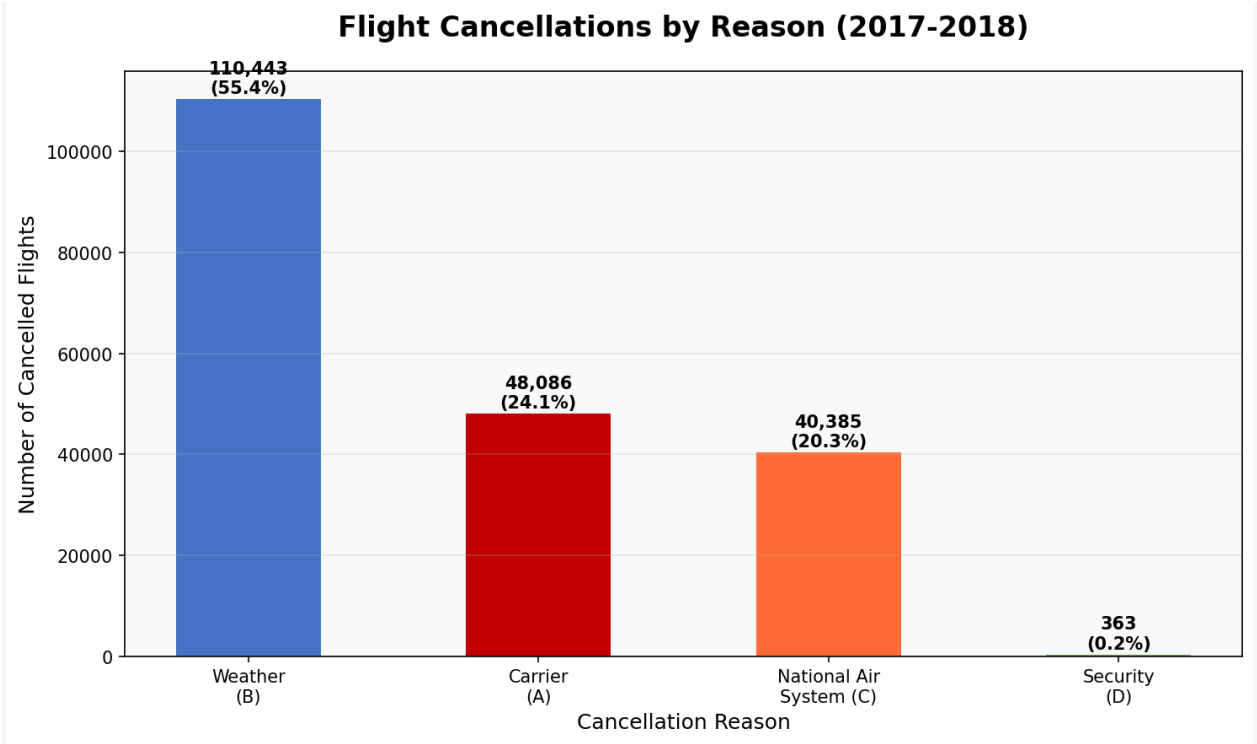


Figure 11: Flight cancellations by reason (2017-2018). Weather accounts for 55.4% of cancellations despite causing only 5.1% of delay minutes.

4.6 Analysis 6 — Monthly Average Arrival Delay Trend

ANALYSIS 6: Monthly Average Arrival Delay Trend

month	avg_delay_min
1	39.21
2	35.74
3	32.9
4	38.31
5	38.28
6	41.25
7	42.03
8	41.15
9	34.59
10	32.28
11	33.14
12	35.58

Month	Avg Delay (min)	Pattern
January	39.21	Winter — elevated
February	35.74	Winter — moderate
March	32.90	Spring — improving
April	38.31	Spring storms rising

May	38.28	Pre-summer
June	41.25	Summer peak begins
July	42.03	Summer peak — HIGHEST
August	41.15	Summer still elevated
September	34.59	Post-summer drop
October	32.28	Autumn — LOWEST
November	33.14	Pre-holiday
December	35.58	Holiday season

The monthly trend reveals a clear dual-peak pattern that aligns well with what we know about US weather and travel patterns. July is the worst month at 42.03 minutes average, it is a combination of peak summer travel volumes, afternoon thunderstorm season across the eastern US, and maximum airspace congestion. October is the best at 32.28 minutes, sitting comfortably between the end of hurricane season and the start of winter storms. What we found interesting about December is how modest the delays are (35.58 minutes) despite being one of the busiest travel periods of the year. We think this probably reflects airlines building extra schedule buffer time into their winter timetables — effectively absorbing expected delays into the planned arrival times rather than recording them as delays.

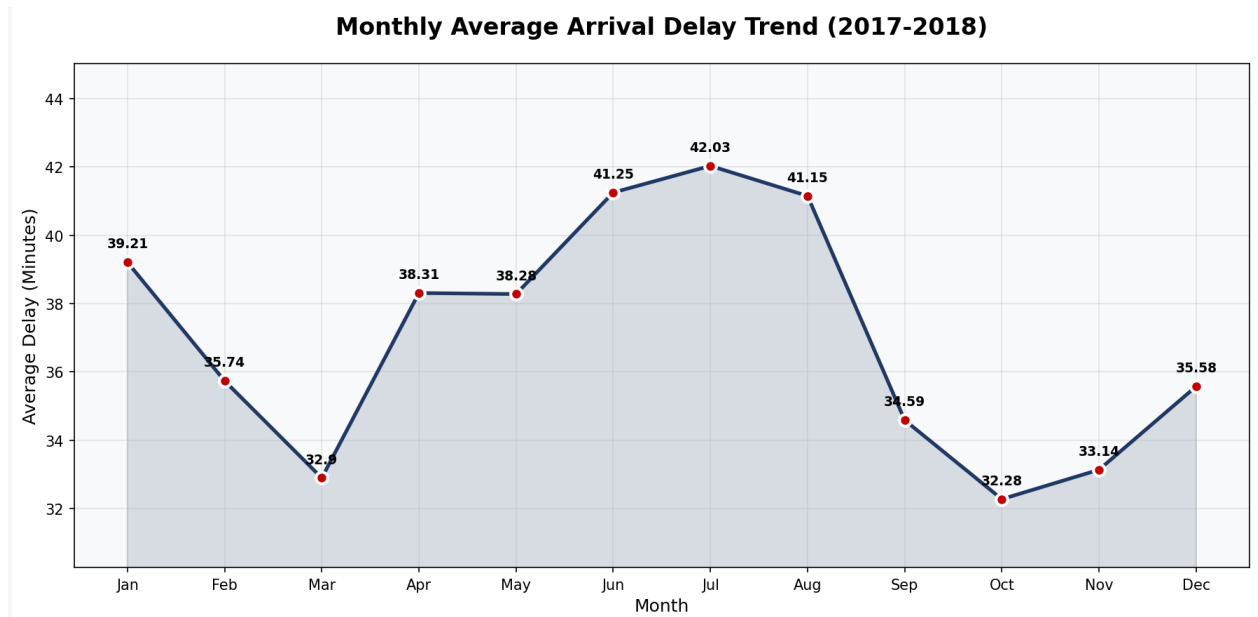


Figure 12: Monthly average arrival delay trend (2017-2018). Clear dual-peak pattern with July highest at 42.03 min and October lowest at 32.28 min.

4.7 Analysis 7 — Top 3 Most Delayed Routes per Airline (Window Function)

ANALYSIS 7: Top 3 Most Delayed Routes per Airline (Window Function)

```

=====
+-----+-----+-----+-----+-----+
|OP_CARRIER|ORIGIN|DEST|avg_delay_min|flight_count|rank|
+-----+-----+-----+-----+-----+
| 9E | MSY | JFK | 95.19 | 106 | 1 |
| 9E | JFK | PHL | 93.87 | 52 | 2 |
| 9E | IND | LGA | 91.36 | 156 | 3 |
| AA | EGE | JFK | 127.78 | 60 | 1 |
| AA | EGE | MIA | 105.45 | 82 | 2 |
| AA | ANC | DFW | 104.88 | 107 | 3 |
| AS | EWR | SFO | 55.34 | 443 | 1 |
| AS | DCA | SFO | 54.18 | 124 | 2 |
| AS | BWI | SFO | 53.48 | 101 | 3 |
| B6 | ACK | JFK | 77.47 | 395 | 1 |
| B6 | ORD | JFK | 67.8 | 982 | 2 |
| B6 | FLL | SYR | 67.33 | 136 | 3 |
| DL | BGR | JFK | 100.62 | 65 | 1 |
| DL | SRQ | LGA | 98.35 | 89 | 2 |
| DL | MSP | ATW | 94.38 | 81 | 3 |
| EV | ROW | DFW | 129.1 | 67 | 1 |
| EV | TVC | EWR | 122.4 | 88 | 2 |
| EV | TYS | LGA | 120.54 | 74 | 3 |
| F9 | AUS | SJC | 81.81 | 62 | 1 |
| F9 | TTN | TPA | 76.47 | 133 | 2 |
| F9 | TPA | TTN | 75.31 | 166 | 3 |
| G4 | IND | SFB | 81.8 | 109 | 1 |
| G4 | PBG | FLL | 75.1 | 61 | 2 |
| G4 | PGD | CVG | 74.95 | 142 | 3 |
| HA | PPG | HNL | 81.66 | 105 | 1 |
| HA | PHX | HNL | 54.31 | 219 | 2 |
| HA | HNL | PPG | 53.75 | 114 | 3 |
| MQ | MSO | DFW | 62.21 | 57 | 1 |
| MQ | BWI | ORD | 60.02 | 369 | 2 |
| MQ | MQT | ORD | 59.01 | 190 | 3 |
+-----+-----+-----+-----+-----+
only showing top 30 rows

```

Carrier	Route	Avg Delay (min)	Flights Analysed
EV (ExpressJet)	ROW → DFW	129.10	67
AA (American)	EGE → JFK	127.78	60
EV (ExpressJet)	TVC → EWR	122.40	88
DL (Delta)	BGR → JFK	100.62	65
9E (Endeavor)	MSY → JFK	95.19	106
OO (SkyWest)	Single flight max delay	2,692 min	—

The window function analysis produced some of the most interesting findings in the entire project. Routes into New York area airports, specifically JFK, LGA, and EWR, appear repeatedly across multiple airlines, confirming that New York airspace is the most congested in the country. American's EGE to JFK route averaged 127.78 minutes of delay. The compounding effect of a challenging mountain origin airport combined with a transcontinental flight into the busiest airspace in the US explains why this route performs so poorly. ExpressJet's ROW to DFW route from Roswell, New Mexico to Dallas averaged 129.10 minutes, the highest of any route in the dataset. We think this is a result of extreme summer heat at Roswell's high-desert elevation affecting aircraft performance and extending ground times on the ground before departure.

The window function was the right tool here because it allowed us to preserve individual route-level detail including origin, destination, average delay, and flight count, while adding a rank column relative to each airline. Using groupBy instead would have collapsed everything into one summary row per airline, and we would have lost all the route-level specifics that make this analysis meaningful.

4.8 Year-on-Year Comparison: 2017 vs 2018

year	total_flights	delayed_flights	avg_delay_min	cancellations	cancel_rate_pct
2017	5674621	1944547	36.83	82693	1.46
2018	7213446	2516050	37.73	116584	1.62

year	carrier_min	weather_min	nas_min	late_aircraft_min
2017	2.0516702E7	2799759.0	1.6500787E7	2.590507E7
2018	2.6316981E7	4919075.0	2.1488435E7	3.4689058E7

year	OP_CARRIER	avg_delay_min	delayed_flights	rank
2017	EV	51.72	109946	1
2017	B6	50.02	116031	2
2017	OO	44.99	239368	3
2017	NK	43.83	51420	4
2017	UA	42.64	169779	5
2018	EV	51.69	68244	1
2018	F9	50.13	52413	2
2018	9E	49.17	73316	3
2018	B6	46.49	126936	4
2018	G4	44.93	37177	5

Metric	2017	2018	Change
Total flights	5,674,621	7,213,446	+27.1%
Delayed flights	1,944,547	2,516,050	+29.4%
Avg delay (delayed flights)	36.83 min	37.73 min	+0.9 min
Cancellations	82,693	116,584	+41.0%
Cancellation rate	1.46%	1.62%	+0.16%

Across every metric we measured, the U.S. airline industry's operational performance worsened significantly in 2018. The total number of flights increased by 27.1%, reflecting strong demand growth driven by the economic expansion that began in 2016—but the increase in flight delays was even higher, at 29.4%, indicating that the airline network is under growing pressure from the surge in passenger traffic. The number of flight cancellations jumped from 82,693 to 116,584, a 41% increase, which is a shocking deterioration over just one year.

Furthermore, the delay cause breakdown tells us an even more striking story, the weather delay minutes nearly doubled from 2.8 million to 4.9 million this is a 75.7% increase year-on-year. This likely reflects the particularly active 2018 hurricane season and major winter storm events that year. Late aircraft cascading delays also grew substantially, from 25.9 million to 34.7 million minutes (+33.9%), consistent with the overall increase in flight operations straining the network's ability to recover from disruptions.

Looking at airline performance year-on-year, ExpressJet (EV) was the only airline to appear as the worst performer in both 2017 (51.72 min average) and 2018 (51.69 min average), this consistency

showing that it is something about structural operational problems rather than a one-off bad year. Meanwhile, JetBlue improved from 50.02 to 46.49 minutes but remained in the worst five both years. Frontier (F9) entered the worst five in 2018 at 50.13 minutes, a result consistent with the rapid expansion of its network that year, which may have exceeded its operational capacity.

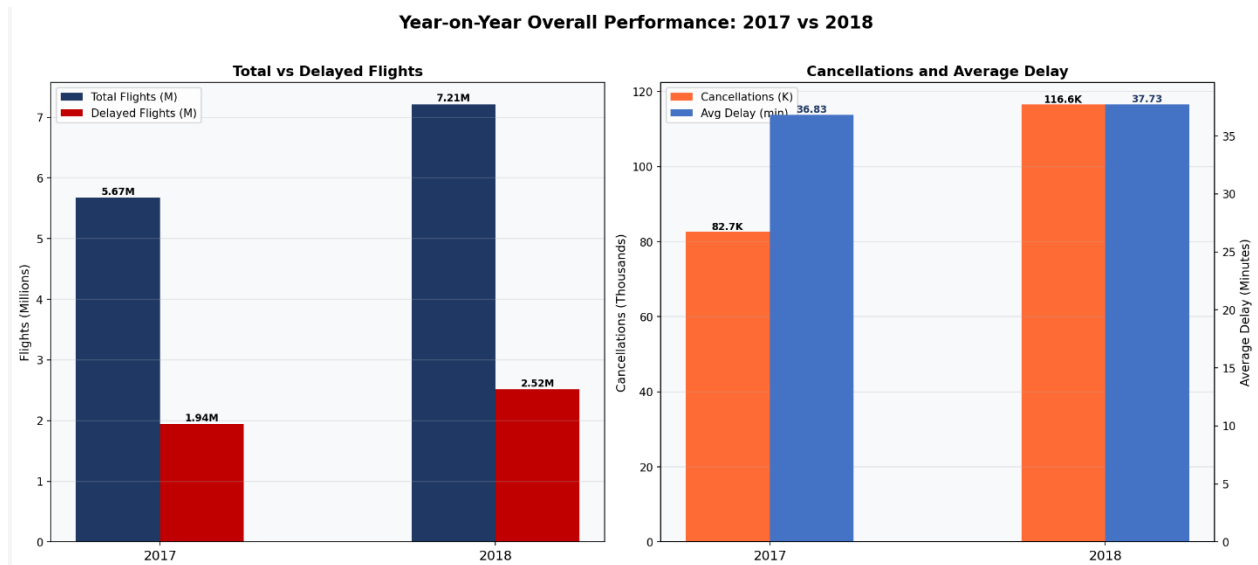


Figure 13: Year-on-year overall performance comparison. Every metric worsened in 2018, with cancellations up 41% and delayed flights up 29.4%.

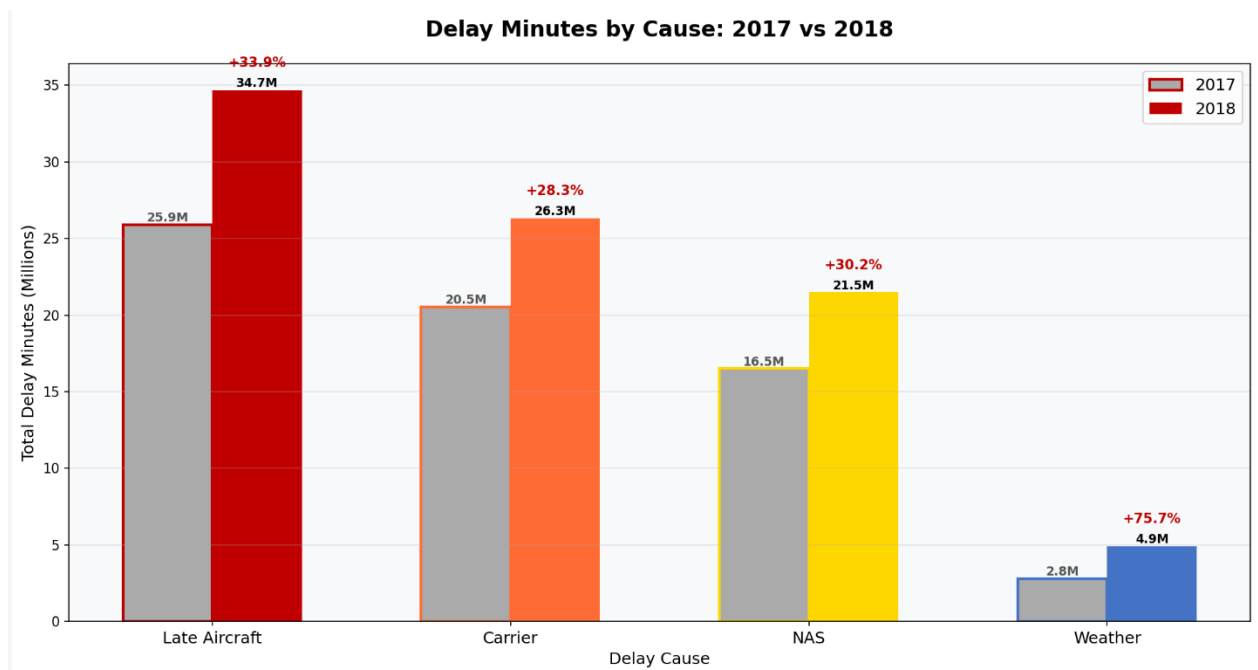


Figure 14: Delay causes comparison by year. Weather delay minutes increased 75.7% from 2017 to 2018 — the largest proportional increase of any cause.

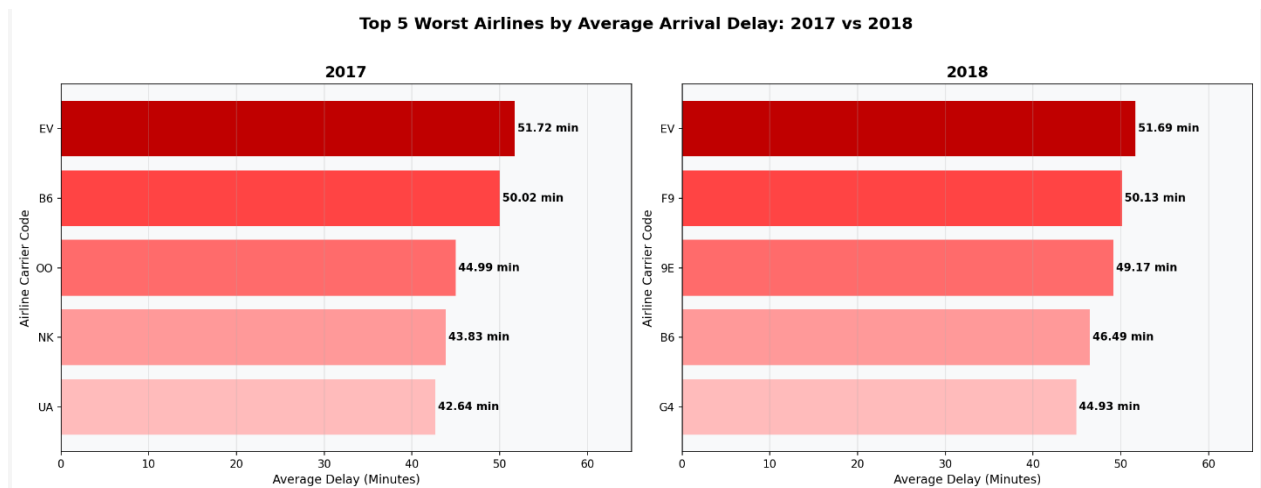


Figure 15: Top 5 worst airlines by average delay per year. ExpressJet (EV) is the only airline appearing worst in both 2017 and 2018.

5.0 Individual Reflection

5.1 Lee Zong Xun 22083844

Throughout this project, my main responsibility was mostly the technical side, such as setting up and managing the EMR cluster, uploading data to HDFS, writing all 7 analysis and visualization scripts, and then debugging the different issues that popped up along the way. To be honest, while coding and debugging, I mostly relied on the Claude AI tool for help, but I always verify the output myself and make sure I understand it thoroughly before I can put it to use. The most memorable thing was when our first EMR cluster terminated unexpectedly, and all the HDFS data disappeared with it, which meant we had to restart completely from scratch. It was honestly frustrating at the time, but it taught me something I will not forget about it, which is in the cloud environments, anything stored only on the cluster nodes is temporary. After that, I got into the habit of backing up outputs to S3 immediately after every Spark run, because that is basically the cloud native pattern that our labs are trying to help us understand in depth.

From our analysis, the result that genuinely surprised me the most was from Analysis 3 — the late aircraft cascading delays accounted for 39.9% of all delay minutes, while weather only caused 5.1%. Before running the analysis, I would have predicted weather as the dominant factor, pretty confidently too, since that's what passengers and airlines usually say. But once I saw the actual numbers, that assumption flipped entirely. Furthermore, I run the same analyses across three Spark API layers — RDD, DataFrame, and SQL — and seeing all three produce byte-for-byte identical results, really sharpened my understanding of how the Catalyst optimizer works. It doesn't matter which "surface" you use to write the query; the physical execution plan underneath ends up being the same.

5.2 Bernice Wong Jian Xuan 22056998

In this project, my core responsibilities focus on the logical construction of research problems, the overall planning of analytical reports, and the in-depth distillation of business insights. My main goal is to transform Spark's technology output into easy-to-understand and operational business insights, so as to bridge the information gap between bottom-level technology realisation and high-level business decision-making.

In the collaboration with my technical teammate Zong Xun, I have a concrete understanding of the system efficiency of distributed computing. The concept of "multi-node parallel processing" previously encountered in classroom teaching is relatively abstract. In this practice, the Spark engine built on the EMR cluster took only about 3 minutes to complete the cleaning and complex aggregation of more than 12.88 million flight history records. This measured performance intuitively shows the significant advantages of parallel architecture in massive data processing.

In the data quality assessment, a key finding profoundly changed my understanding of data interpretation: the data shows that 46.6% of the delay records did not clearly mark the reason for the delay. At first, this result is easily attributed to data collection defects or missing systems, but after investigating the industry background, it is found that the root cause lies in BTS's standard escalation policy that is, only the cause is attributed to flights with delays of 15 minutes or more. This phenomenon fully illustrates that data interpretation cannot be limited to statistical

appearances but must be fully combined with the business rules and policy constraints of the data collection end. Lack of Domain Knowledge data analysis is very easy to misjudge the actual situation. In addition, the annual year-on-year analysis led by me constitutes the key narrative dimension of the whole report. By expanding the analytical perspective from static data to dynamic evolution across years, the report shows a stronger data narrative. The comparative analysis clearly reveals the evolution trajectory of the route system in 2018: with the continuous growth of flight volume and the frequent occurrence of bad weather (such as the hurricane season), various delay indicators have shown a trend of overall deterioration, dynamically reflecting the pressure-bearing state of the aviation network under multiple external pressures.

6.0 References

Bureau of Transportation Statistics. (2019). *Airline delay and cancellation data, 2009–2018* [Dataset]. Kaggle. <https://www.kaggle.com/datasets/yuanyuwendymu/airline-delay-and-cancellation-data-2009-2018>

Amazon Web Services. (2024). *Amazon EMR documentation*. <https://docs.aws.amazon.com/emr/>

Apache Software Foundation. (2024). *Apache Spark documentation*.
<https://spark.apache.org/docs/latest/>

U.S. Department of Transportation, Bureau of Transportation Statistics. (2024). *Airline on-time performance and causes of flight delays*. <https://www.bts.gov/explore-topics-and-geography/topics/airline-time-performance-and-causes-flight-delays>